TD : Récursivité

Terminale NSI — Durée : 2 heures

Consignes : tous les programmes sont à écrire en Python et à tester dans Thonny. Deux exercices utilisent en plus l'outil en ligne Python Tutor, qui permet de visualiser l'exécution pas à pas des programmes. Les exercices marqués (⋆) sont facultatifs.

Rappels

Une fonction est **récursive** si elle s'appelle elle-même. Elle doit contenir :

- un ou plusieurs **cas de base** (qui ne provoquent pas d'appel récursif) ;
- des **appels récursifs** dont les paramètres se rapprochent d'un cas de base (c'est ce qui garantit la **terminaison**).

La récursivité est **multiple** quand le corps de la fonction contient plusieurs appels récursifs. Elle est **terminale** quand l'appel récursif est la **dernière** opération effectuée : il ne reste rien à faire après le retour de l'appel.

1 Partie A – Récursivité simple

Exercice 1 — Que fait cette fonction ?

```
def mystere1(a, b):
    if b == 0:
        return 0
    return a + mystere1(a, b - 1)
```

1. Sans ordinateur, dérouler l'exécution de `mystere1(3, 4)` : écrire la suite des appels (la "descente"), puis les valeurs renvoyées une à une (la "remontée").
2. Que calcule cette fonction ?
3. Quel est le cas de base ? Quelle quantité décroît à chaque appel ?
4. Vérifier votre résultat en exécutant la fonction dans Thonny.

Exercice 2 — Que fait cette fonction ?

```
def mystere2(n):
    if n == 0:
        return 0
    return n % 2 + 10 * mystere2(n // 2)
```

1. Calculer à la main `mystere2(6)` puis `mystere2(13)`, en présentant les appels successifs.
2. Que calcule cette fonction ?
3. Justifier qu'elle se termine toujours (quelle quantité décroît à chaque appel ?).
4. Vérifier vos résultats dans Thonny.

Exercice 3 — Somme des entiers

On veut calculer $S(n) = 1 + 2 + \cdots + n$ pour $n \in \mathbb{N}$.

1. Donner une relation entre $S(n)$ et $S(n-1)$, puis la valeur de $S(0)$.
2. Écrire une fonction récursive `somme(n)` qui renvoie $S(n)$.
3. Dérouler à la main l'exécution de `somme(4)`.

4. Saisir votre fonction dans **Python Tutor** (`pythontutor.com`) (accompagnée de l'appel `print(somme(4))`) puis l'exécuter pas à pas. Décrire ce qui se passe dans la pile d'appels (zone "Frames") pendant la descente, puis pendant la remontée.
5. Que se passe-t-il si on appelle `somme(-1)` ? Pourquoi ?

Exercice 4 — Puissance

1. Écrire une fonction récursive `puissance(x, n)` qui renvoie x^n (avec $n \in \mathbb{N}$) **sans utiliser** l'opérateur `**`.
2. Combien de multiplications sont effectuées pour calculer `puissance(2, 10)` ?

(⋆) En utilisant le fait que $x^n = (x^{n/2})^2$ si n est pair et $x^n = x \cdot (x^{(n-1)/2})^2$ sinon, écrire une version qui effectue beaucoup moins de multiplications.

Exercice 5 — Manipulation de chaînes

On n'utilise ni `len`, ni `reversed`. On rappelle que `s[1:]` est la chaîne `s` privée de son premier caractère.

1. Écrire `longueur(s)` qui renvoie le nombre de caractères de `s`.
2. Écrire `inverse(s)` qui renvoie la chaîne `s` à l'envers.
3. Écrire `est_palindrome(s)` qui renvoie `True` si `s` se lit de la même façon dans les deux sens (on pourra utiliser `s[-1]` et `s[1:-1]`).

Exercice 6 — Somme des chiffres

Écrire une fonction récursive `somme_chiffres(n)` qui renvoie la somme des chiffres de l'entier naturel `n` (par exemple `somme_chiffres(2026)` renvoie `10`). On pourra utiliser `n % 10` et `n // 10`.

2 Partie B – Récursivité multiple

Exercice 7 — Arbre avec la tortue

Le module `turtle` permet de dessiner en déplaçant une "tortue". On considère le programme suivant (à saisir dans Thonny) :

```
import turtle

def arbre(longueur, niveau):
        if niveau == 0:
                return
        turtle.forward(longueur)
        turtle.left(30)
        arbre(longueur * 0.7, niveau - 1)
        turtle.right(60)
        arbre(longueur * 0.7, niveau - 1)
        turtle.left(30)
        turtle.backward(longueur)

turtle.speed(0)
turtle.left(90) # la tortue regarde vers le haut
arbre(100, 3)
turtle.done()
```

1. Exécuter le programme avec `niveau` égal à 1, puis 2, puis 3, puis 4 (à la place du 3 dans `arbre(100, 3)`). Pour chaque valeur, compter le nombre de segments tracés. Combien de segments seront tracés pour une exécution de `niveau` n ($n \in \mathbb{N}$) ?

2. Quel est le cas de base ? Que dessine l'appel `arbre(longueur, 1)` ?
3. Les trois dernières lignes de la fonction ramènent la tortue à son point de départ, dans sa direction de départ. Pourquoi est-ce indispensable pour que le dessin soit correct ?
4. Modifier les valeurs 0,7 ; 30 et 60 pour obtenir un arbre différent.

Exercice 8 — Suite de Fibonacci

On définit $F_0 = 0$, $F_1 = 1$ et $F_n = F_{n-1} + F_{n-2}$ pour $n \geq 2$.

1. Compléter la fonction récursive `fib(n : int)-> int` qui renvoie le terme de rang *n* de la suite de Fibonacci.

```
def fib(n : int) -> int:
    if n < 2:
        return ..................
    else:
        return ..................
```

2. Dessiner l'arbre des appels de `fib(4)`. Combien d'appels de la fonction `fib` (y compris l'appel initial `fib(5)` et ceux des cas de base) sont effectués pour calculer `fib(5)` ?
3. Saisir `fib` dans **Python Tutor** (`pythontutor.com`) avec l'appel `print(fib(4))`, puis l'exécuter pas à pas. Vérifier votre arbre d'appels et repérer les calculs qui sont répétés.
4. Mesurer avec le module `time` la durée de `fib(30)` puis `fib(35)`. Que constate-t-on ?

   Rappel : la fonction `time.perf_counter()` renvoie l'instant courant en secondes. La durée d'un calcul est la différence entre l'instant d'arrivée et l'instant de départ.

```
import time

debut = time.perf_counter()
resultat = fib(30)
fin = time.perf_counter()
print(resultat, "calcule en", fin - debut, "secondes")
```

5. Pourquoi la fonction est-elle si lente ?
   Afin d'améliorer la rapidité d'un calcul de terme, on propose de conserver les valeurs déjà calculées dans un dictionnaire `memo`. Compléter la fonction `fib_memo(n, memo)` qui permet cela :

```
def fib_memo(n, memo=None):
    if memo is None:
        memo = {}
    if n < 2:
        return ...........
    if n not in memo:
        memo[n] = ................................................
    return memo[n]
```

   Mesurer la durée de `fib_memo(30)` puis `fib_memo(35)`. Que constate-t-on ?

3 Partie C – Récursivité terminale

Accumulateur

Pour rendre une fonction récursive terminale, on ajoute souvent un paramètre **accumulateur** qui stocke le résultat partiel : le calcul se fait *avant* l'appel récursif, et le cas de base renvoie l'accumulateur.

Exercice 9 — Reconnaître la récursivité terminale

Pour chacune des fonctions suivantes, indiquer si la récursivité est terminale. Justifier.

```
def f1(n):
    if n == 0:
        return 1
    return n * f1(n - 1)

def f2(a, b):
    if b == 0:
        return a
    return f2(b, a % b)

def f3(l):
    if l == []:
        return 0
    return l[0] + f3(l[1:])

def f4(n):
    if n > 0:
        print(n)
        f4(n - 1)

def f5(n):
    if n < 2:
        return n
    return f5(n - 1) + f5(n - 2)
```

Exercice 10 — Fonction mystère terminale

```
def mystere3(n, acc=0):
        if n == 0:
                return acc
        return mystere3(n // 10, 10 * acc + n % 10)
```

1. Sans ordinateur, dérouler l'exécution de `mystere3(1234)` en notant à chaque appel les valeurs de `n` et `acc`.
2. Que calcule cette fonction ? Quel est le rôle de l'accumulateur `acc` ? On pourra vérifier la réponse en calculant à la main `mystere3(506)` puis `mystere3(120)`.
3. Justifier que la récursivité est terminale.

Exercice 11 — Version terminale avec accumulateur

On reprend la fonction `somme` de l'exercice 3.

1. Pourquoi la récursivité de `somme` n'est-elle pas terminale ? Écrire une version récursive terminale `somme_acc(n, acc=0)` qui renvoie $1 + 2 + \cdots + n$, en utilisant l'accumulateur `acc` pour stocker le résultat partiel.
2. Dérouler à la main `somme_acc(4)` en notant à chaque appel les valeurs de `n` et `acc`.

Exercice 12 — Fibonacci terminal

On utilise deux accumulateurs `a` et `b` qui contiennent deux termes consécutifs de la suite.

1. Écrire `fib_term(n, a=0, b=1)` récursive terminale qui renvoie F_n.
2. Dérouler `fib_term(5)`. Combien d'appels sont effectués ? Comparer avec `fib(5)` de l'exercice 8.